PROJECT-I · REVIEW-2 REPORT

AI-Based Video Summarization System

Using Computer Vision and Natural Language Processing

Understand any video without watching it.

Course Code	BCSE497J — Project-I
Programme	Bachelor of Technology (B.Tech.)
Academic Year	2026–2027
Review	Review-2, Second Week of August 2026 (tentative)
Submitted on	9 September 2026

SUBMITTED BY

Jaivardhan Singh
23BCI0036

Sujay Chand
23BCE0217

Harshit Raj
23BCE0190

UNDER THE GUIDANCE OF

Dr. Deepa K
School of Computer Science and Engineering (SCOPE)

Contents

1	Project Title	3
2	Abstract	3
3	Literature Survey	3
4	Objectives	4
5	Methodology	5
6	Implementation Progress	7
7	Demonstration	9
8	Completion of the Proposed Implementation	9
9	Basis for Consideration by the Review Panel	10
	References	10

1. Project Title

AI-Based Video Summarization System Using Computer Vision and Natural Language Processing — an interactive multimodal assistant that ingests any video, transcribes and watches it, and returns a teachable explanation, a navigable chapter timeline, a highlight reel, and a grounded question-answering interface over the video's content.

2. Abstract

Video is now the dominant medium for lectures, meetings and technical explanation, but it is the least searchable one: extracting five minutes of relevant content from a sixty-minute recording still requires watching the recording. Existing work addresses parts of this problem in isolation. Extractive video-summarization models select which frames matter but never say what they contain; abstractive text summarizers compress a transcript into an abstract that names a topic without teaching it; and video question-answering systems treat the transcript as flat text, discarding the slides, diagrams and code that carry most of the information in an instructional video.

This project builds a single system that produces an explanation a reader can learn from without watching the source. A seven-stage pipeline (M1–M7) ingests the video audio-first, transcribes it with a silence-split parallel Whisper implementation, segments it into scenes and extracts keyframes, objects and on-screen text, fuses the audio and visual tracks onto one timeline, and hands the whole transcript together with selected keyframes to a multimodal large language model under a constrained output schema. The result is rendered as a section-by-section walkthrough with timestamps, a glossary, chapter navigation, per-slide notes and, for programming videos, the algorithm with implementations in four languages. A FAISS index over the transcript supports grounded follow-up questions with cited timestamps.

Two components are contributed beyond the assembly of existing models. First, an adaptive budget planner profiles the host machine, self-calibrates from every completed run, and selects the heaviest model configuration predicted to finish inside a stated wall-clock target — making processing time a published guarantee rather than an outcome. Second, a distributed SLA-constrained scheduler learns each worker class's cost model online and routes each job to the cheapest class still predicted to meet its deadline. On a 43.5-minute lecture the complete pipeline runs in 6.3 minutes on a 16-core CPU-only machine with no GPU. The learned scene-importance model reaches an F1 of 0.265 on TVSum against a human leave-one-out agreement of 0.260 measured in the same pipeline.

3. Literature Survey

Four lines of work bear directly on this problem. Each is strong at its own task; none of them, alone or in combination, produces something a reader can learn from.

Area	Representative work	What it contributes	What it leaves undone
Extractive video summarization	vsLSTM / dppLSTM [12]; DR-DSN [13]; surveys [1], [2]; unsupervised [5]; attention fusion [6]	Learns which frames or shots matter and selects a keyframe subset, benchmarked on SumMe [3] and TVSum [4].	Selection is not explanation. The output is a shorter video, so the viewer must still watch it to learn anything.

Area	Representative work	What it contributes	What it leaves undone
Abstractive text summarization	BART [10]; PEGASUS; DistilBART; the Transformer [7]	Compresses long text into fluent prose and is the standard approach to summarizing a transcript.	Fixed encoder windows force lossy chunking, and on spoken, often machine-translated text the output degrades into disjointed fragments. Nothing visual is used.
Speech and visual perception	Whisper [8]; YOLO [9]; CLIP [14]	Robust transcription, object detection and joint image-text embeddings, all available pre-trained.	These are components, not a system. None of them decides what matters or how the pieces should be combined into an explanation.
Video question answering and retrieval	Retrieval-augmented generation [11]; Sentence-BERT [15]; FAISS [16]	Grounds an answer in retrieved passages and cites its source.	Treats the transcript as flat text: on-screen material is dropped, and there is no persistent understanding of the video to build on.
Evaluation methodology	Otani et al. [17]; Potapov et al. [18]	Shows that randomly generated summaries score close to state of the art under the standard F1 protocol, and proposes rank correlation against each annotator instead.	Much of the literature still reports the F1 figure, so published numbers are not comparable across differing pipelines.

Table 1. Prior work and the gap each line of work leaves open.

3.1 Research gaps identified

- **Summaries are not teachable.** A hundred-word abstract of a sixty-minute lecture names the topic; the reader must still watch the video to understand it. No existing pipeline sets out to teach the content back.
- **On-screen material is discarded.** In lectures and technical talks most of the information lives in diagrams, derivations and code on the screen. Transcript-only pipelines never look at it.
- **Quality collapses with length.** A fixed output budget spread across a longer video thins the explanation until it is useless, so systems that work on a ten-minute clip fail on a three-hour lecture.
- **No wall-clock guarantee.** Pipelines are built around a fixed set of models, so processing time scales freely with input length and nothing in the system is accountable to a deadline.

4. Objectives

Each objective answers one of the gaps identified above.

#	Objective	Addresses	Realised as
1	Produce a **teachable explanation**: teach the video back section by section, filling gaps from standard knowledge and marking clearly what was added.	Gap 1	Stage M5', constrained-schema LLM understanding
2	**Read what is on the screen**: extract every slide, diagram and board as a keyframe with its text, and explain each one.	Gap 2	Stage M3 (keyframes, OCR) and M4 (fusion)

#	Objective	Addresses	Realised as
3	**Hold quality at any length**: keep explanation density constant from ten minutes to twelve hours.	Gap 3	Windowed understanding with a reduce pass
4	**Hold a wall-clock budget**: choose the heaviest configuration that still finishes inside a stated time target on the machine at hand.	Gap 4	Adaptive budget planner and SLA scheduler
5	**Answer doubts conversationally**: ground answers in the video, cite timestamps, and state plainly when an answer goes beyond it.	Gaps 1, 2	Stage M6, retrieval-augmented tutor

Table 2. Objectives mapped to the identified gaps and to the modules that implement them.

5. Methodology

5.1 Pipeline architecture

The system is organised as seven modules. Every module writes its output as a cached artefact that the next module reads, so any stage can be inspected, resumed after a failure, or skipped entirely when its artefact already exists. Stages M2 and M3 have no data dependency on each other and are executed concurrently; M4 is the first stage that requires both.

Stage	Function	Principal technique	Output artefact
M1 Ingest	Fetch the source and extract audio	yt-dlp with eight concurrent fragments; audio-only stream downloaded first so transcription can begin before the video arrives; ffmpeg to 16 kHz mono	audio.wav, meta.json
M2 Transcribe	Speech to timestamped text	faster-whisper (CTranslate2, int8). Audio split at silences detected by ffmpeg and transcribed by twelve parallel worker processes; language detected from three probes spread across the audio	transcript.json
M3 Vision	Analyse what is on the screen	Scene segmentation (content-based below five minutes, CLIP-embedding sampling above it); keyframe selection with CLIP de-duplication; YOLOv8n object labels; EasyOCR on-screen text	scenes.json
M4 Fuse	Align the audio and visual tracks	Interval join of transcript segments onto scene time ranges; no model is involved	fused.json
M5' Understand	Produce the explanation	Whole transcript plus up to twelve keyframes sent to a multimodal LLM under a constrained JSON schema; windowed into 25-minute slices with a reduce pass beyond 45 minutes	understanding.json, summary.json
M6 Index	Make the video answerable	400-token chunks with 80-token overlap, MiniLM-L6-v2 embeddings, FAISS index; the tutor is additionally given the understanding document	index.faiss

Stage	Function	Principal technique	Output artefact
M7 Highlight	Score scene importance	CLIP features at 2 fps into a bidirectional GRU with an MLP head; Kernel Temporal Segmentation for shots; knapsack selection	importance scores, highlight reel

Table 3. The seven pipeline stages, their techniques and their artefacts.

5.2 Adaptive budget planner

Rather than fixing a model set and accepting whatever runtime results, the pipeline is given a wall-clock target and a ladder of configurations for each stage. Before a run begins the planner probes the host (core count and a timed matrix benchmark), loads a calibration profile keyed by usable cores and worker class, divides the budget along the critical path, and walks each ladder down until the predicted total fits. After the run, what each stage actually cost is folded back into the profile using an asymmetric exponential moving average, so a slow surprise is believed faster than a fast one and the planner degrades quality before it overruns its deadline.

5.3 Understanding layer

The understanding stage is what separates this system from a transcript summarizer. A context window of roughly one million tokens allows the entire transcript to be processed in a single call, so nothing is lost at a chunk boundary. Keyframes accompany the text, letting the model read diagrams and handwriting that OCR cannot recover. The kind of video — lecture, coding, meeting, interview, talk, news or documentary — is detected first and reshapes both the prompt and the output sections: a programming video additionally yields the algorithm, its intuition, its time and space complexity and implementations in several languages, while a meeting yields decisions and action items. Every generated section carries a flag recording whether it came from the video or from the model's own knowledge, so a reader can always tell the two apart. Where no provider is configured the system falls back to a local BART map-reduce with flan-t5 chapter titles, which runs entirely offline.

5.4 Distributed SLA-constrained scheduler

This is the novel contribution of the project. Each registered worker class maintains its own online cost model. On submission the scheduler (i) predicts the job's cost on every class, (ii) routes it to the cheapest class still predicted to meet the wall-clock SLA, (iii) folds the observed timing back into that class's model on completion, and (iv) explores thinly-measured classes only while they remain predicted to meet the SLA — exploration that cannot break the promise.

5.5 Technology stack

Layer	Components	Note
Perception	faster-whisper (int8) · YOLOv8n at 416 px · CLIP ViT-B/32 · EasyOCR · PySceneDetect	All pre-trained, all CPU-only
Understanding	Gemini 2.5 Flash · FAISS + MiniLM-L6-v2 · DistilBART / flan-t5 fallback	The only remote dependency
Learned model	CLIP features into BiGRU + MLP (PyTorch); Kernel Temporal Segmentation	The one model trained in this project

Layer	Components	Note
Service	FastAPI + SQLAlchemy · Redis + RQ workers · Docker Compose · Server-Sent Events	Asynchronous job model
Interface	Next.js 15 + Tailwind (light and dark) · Streamlit	Live per-stage progress

Table 4. Technology stack. Every perception model is used pre-trained; only the M7 scene-importance model was trained for this project.

6. Implementation Progress

All seven modules are complete and running end to end. What remains is deployment and documentation, not core function.

6.1 Status

Component	State
M1–M7 pipeline, end to end, with per-stage caching	**Complete**
LLM understanding, with windowing for long videos	**Complete**
Tutor question answering: conversation history, cited timestamps	**Complete**
M7 scene-importance model trained and driving the highlight reel	**Complete**
FastAPI service: jobs, SSE progress, uploads, authentication, cancellation	**Complete**
Next.js frontend: light and dark themes, live progress, library, Q&A, scenes, report export	**Complete**
Distributed SLA scheduler, exercised on a real two-class worker fleet	**Complete**
Cloud deployment (configuration written; Vercel plus a container host)	**In progress**
Final project report and wider evaluation across video categories	**In progress**

Table 5. Implementation status at Review-2.

6.2 Codebase

10,790	**1,705**	**50**	**40**
lines of Python (core, server, ml, ui)	lines of TypeScript (Next.js frontend)	automated tests, all passing	commits on the main branch

6.3 Measured processing performance

Measured on the development machine: a 16-core CPU-only laptop with no GPU. The 43.5-minute figure is an end-to-end measurement; the remaining rows are planner predictions on the same machine, shown against the processing-time targets set for the product.

Video length	Stage breakdown	Total	Target
43.5 min *(measured)*	ingest 37.8 s · analyze 138.2 s · understand 92.2 s · index 23.4 s	**6.3 min**	—
30 min *(predicted)*	—	**3.7 min**	4–5 min
1 hour *(predicted)*	—	**7.3 min**	15–20 min
2 hours *(predicted)*	—	**14.7 min**	30–35 min
3 hours *(predicted)*	—	**22.0 min**	30–35 min

Table 6. Processing time against the published targets. Transcription cost fell from 0.135 to 0.058 seconds per second of speech (2.3 times faster) after correcting thread oversubscription across the concurrent stages. Non-English videos require translation and run roughly 1.5 times longer.

6.4 Evaluation of the M7 scene-importance model

Trained on SumMe and TVSum under five-fold, video-disjoint cross-validation over 75 videos. Because absolute F1 is not comparable across pipelines — features, frame sampling and shot segmentation all differ — every baseline below was recomputed inside this pipeline on the same shards.

Knapsack-summary F1	Random	Our model	Human (leave-one-out)	Oracle
TVSum	0.166	**0.265**	0.260	0.406
SumMe	0.368	**0.451**	0.612	0.783

Table 7. On TVSum the model matches how well a single annotator agrees with the others. Rank correlation under the Otani et al. protocol, computed against each annotator separately rather than against their mean: Kendall's tau 0.160, Spearman's rho 0.208.

6.5 Evaluation of the scheduler

Recorded over 20 jobs on a real two-class fleet (8-core and 2-core workers running as separate processes against a Redis queue).

Routing policy	SLA met	Basis
Adaptive routing (this work)	**75.0%**	Measured over 20 real jobs
Always the fast class	**65.0%**	Cost-model prediction
Random choice	**37.5%**	Cost-model prediction
Always the slow class	**10.0%**	Cost-model prediction

Table 8. The honest reading: choosing a class at all moves the met-rate from 10% to 65%, and learned routing then finds the right class. Only the adaptive figure is a measurement; the three baselines are predictions from the same cost models.

7. Demonstration

The following will be shown live to the panel.

#	Step	What the panel will see
1	**Submit a video**	Paste a URL, select the video type and topic, choose a processing profile.
2	**Watch it process**	Live per-stage progress over Server-Sent Events, showing the routed worker class and the predicted completion time.
3	**Read the explanation**	Section-by-section teaching with timestamps, added-context badges marking what came from outside the video, and a glossary.
4	**Algorithm and code**	For a programming video: the algorithm, its complexity, and implementations in Python, Java, C++ and JavaScript.
5	**On-screen material**	The keyframe gallery with extracted text and an explanation of each slide or board.
6	**Ask a doubt**	Follow-up questions answered with conversation history and cited timestamps, including the system stating when an answer goes beyond the video.

Table 9. Demonstration plan. Several videos are already processed and cached, so the demonstration does not depend on the venue network.

8. Completion of the Proposed Implementation

Implementation stands ahead of the 60–75% typically expected at Review-2. All five objectives are functionally met; the outstanding work is deployment, documentation and broader evaluation.

Objective	Status	Evidence
1. Teachable explanation	**Met**	Stage M5' in production; schema-constrained output rendered as a walkthrough with glossary and chapters.
2. Read on-screen material	**Met**	Keyframes with OCR text and per-scene notes; keyframes are also sent to the multimodal model.
3. Hold quality at any length	**Met**	Windowed understanding with a reduce pass; verified on sources up to 11 hours 46 minutes.
4. Hold a wall-clock budget	**Met**	Planner predictions land inside every published target (Table 6); scheduler evidence in Table 8.
5. Conversational doubt clearing	**Met**	Retrieval-augmented tutor with conversation history and calibrated confidence thresholds.

Table 10. Objective completion at Review-2.

8.1 Remaining work and schedule

Task	Detail	Target
Cloud deployment	Frontend to Vercel and the backend container to a managed host; configuration and Dockerfile are written and the memory footprint has been measured at 1.26 GB resident.	September 2026
Wider evaluation	Extend measurement beyond lectures and programming videos to meetings, interviews and documentaries.	October 2026
Final project report	Full report and documentation for the Final Review.	October 2026

Table 11. Work remaining ahead of the Final Review in late October 2026.

9. Basis for Consideration by the Review Panel

The final rubric item is a judgement reserved to the panel. This section sets out the evidence on which that judgement can be made, including what the project does not claim.

9.1 Grounds for consideration

- **A working system, not a prototype.** All seven modules run end to end, behind two complete interfaces, with 50 automated tests passing and per-stage caching that allows a failed run to resume.
- **A contribution beyond assembly.** The SLA-constrained scheduler and the self-calibrating budget planner are original work, and both are evidenced by measurement rather than assertion.
- **Measured, reproducible results.** Every performance figure in this report was measured on one identified machine. Where a number is a prediction rather than a measurement, this report says so.
- **Correct evaluation methodology.** The M7 model is scored under the Otani et al. protocol against each annotator, and every baseline was recomputed inside this pipeline, because published F1 figures are not comparable across differing pipelines.
- **Practical accessibility.** The entire perception stack runs on CPU. No GPU is required at any point, which makes the system deployable on ordinary hardware.

9.2 What is not claimed

- Explanation quality depends on an external LLM provider. The offline fallback works but is substantially weaker.
- All timings are for English sources; translated video runs roughly 1.5 times longer.
- The M7 F1 figures are not comparable with published SumMe and TVSum results, which is precisely why in-pipeline baselines are reported alongside them.
- Three of the four scheduler baselines in Table 8 are cost-model predictions, not measurements.
- Cloud deployment is configured but not yet live.

References

[1] E. Apostolidis, E. Adamantidou, A. I. Metsai, V. Mezaris, and I. Patras, “Video Summarization Using Deep Neural Networks: A Survey,” *Proceedings of the IEEE*, vol. 109, no. 11, pp. 1838–1863, 2021.

[2] P. Saini, K. Kumar, S. Kashid, A. Saini, and A. Negi, “Video summarization using deep learning techniques: a detailed analysis and investigation,” *Artificial Intelligence Review*, vol. 56, pp. 12347–12385, 2023.

[3] M. Gygli, H. Grabner, H. Riemenschneider, and L. Van Gool, “Creating Summaries from User Videos,” in *Proc. ECCV*, 2014, pp. 505–520.

[4] Y. Song, J. Vallmitjana, A. Stent, and A. Jaimes, “TVSum: Summarizing Web Videos Using Titles,” in *Proc. IEEE CVPR*, 2015.

[5] S.-S. Zang, H. Yu, Y. Song, and R. Zeng, “Unsupervised video summarization using deep non-local video summarization networks,” *Neurocomputing*, vol. 519, pp. 26–35, 2023.

[6] Y. Zhang, Y. Liu, and C. Wu, “Attention-guided multi-granularity fusion model for video summarization,” *Expert Systems with Applications*, vol. 249, 123568, 2024.

[7] A. Vaswani et al., “Attention Is All You Need,” in *NeurIPS*, 2017.

[8] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust Speech Recognition via Large-Scale Weak Supervision,” in *Proc. ICML*, 2023, pp. 28492–28518.

[9] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proc. IEEE CVPR*, 2016, pp. 779–788.

[10] M. Lewis et al., “BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension,” in *Proc. ACL*, 2020, pp. 7871–7880.

[11] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *NeurIPS*, 2020.

[12] K. Zhang, W.-L. Chao, F. Sha, and K. Grauman, “Video Summarization with Long Short-Term Memory,” in *Proc. ECCV*, 2016, pp. 766–782.

[13] K. Zhou, Y. Qiao, and T. Xiang, “Deep Reinforcement Learning for Unsupervised Video Summarization with Diversity-Representativeness Reward,” in *Proc. AAAI*, 2018.

[14] A. Radford et al., “Learning Transferable Visual Models From Natural Language Supervision,” in *Proc. ICML*, 2021, pp. 8748–8763.

[15] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[16] J. Johnson, M. Douze, and H. Jégou, “Billion-scale Similarity Search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[17] M. Otani, Y. Nakashima, E. Rahtu, and J. Heikkilä, “Rethinking the Evaluation of Video Summaries,” in *Proc. IEEE CVPR*, 2019, pp. 7596–7604.

[18] D. Potapov, M. Douze, Z. Harchaoui, and C. Schmid, “Category-Specific Video Summarization,” in *Proc. ECCV*, 2014, pp. 540–555.